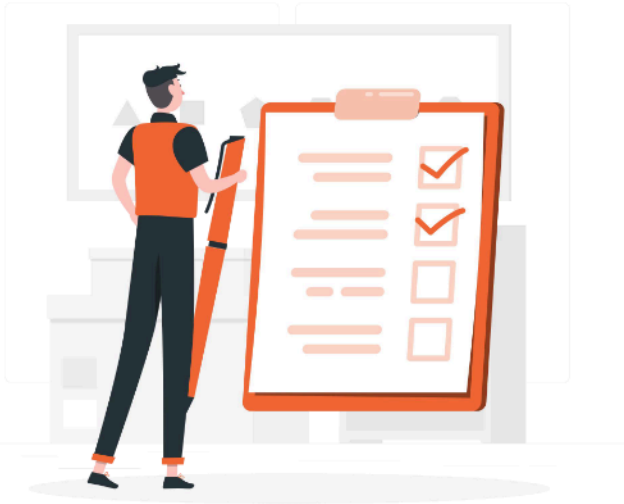
HACKTIV8

Logging, Monitoring, and Analytics

HACKTIV8

Lesson

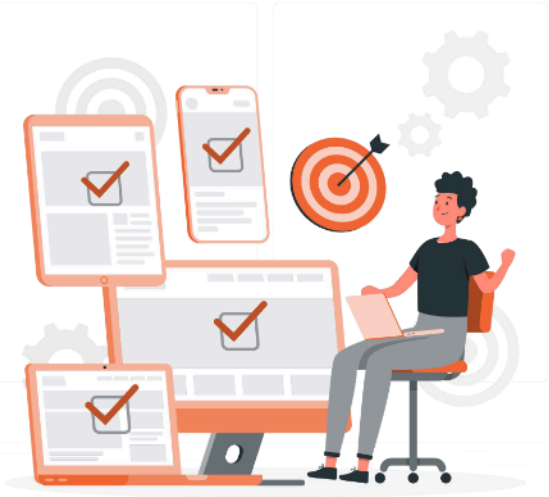
1. Telemetry design
2. Monitoring token usage
3. Grafana/Prometheus style dashboards
4. Operational logging best practice



HACKTIV8

Learning Outcome

1. Explain why AI systems need observability
2. Design telemetry with trace ID propagation
3. Track and govern LLM token usage
4. Build dashboards and alerts with Prometheus and Grafana
5. Write structured logs and correlate them with traces



HACKTIV8

Telemetry Design

HACKTIV8

What is Telemetry in AI Systems

DEFINITION

Telemetry is the stream of measurements, events, and traces your system emits while it runs.

In plain English:

Your system writes a diary about itself while it works. Observability is reading that diary fast enough to react.

Four pillars of observability

Metrics

Numeric, time-series — latency, QPS, errors.

Logs

Event records — what happened in detail.

Traces

How a request flowed across services.

Evals

Quality signals unique to AI systems.

HACKTIV8

Pillars Explained: Same Event, 3 Angles

Metrics

How many? How fast? How often?

Example

```
p95 latency = 820 ms
requests/min = 142
error_rate = 0.4%
```

Use when: dashboards, alerts, trends.

Logs

What exactly happened in this event?

Example

```
{ "trace_id":"abc", "status":"OK",
  "tokens_in":412, "latency_ms":820
}
```

Use when: debugging one specific incident.

Traces

How did a request flow across services?

Example

```
API → Retriever → Reranker
  → LLM → Guardrail → Response
```

Use when: latency is slow somewhere — but where?

HACKTIV8

Why AI Telemetry Differs from Traditional Apps

Outputs aren't deterministic

Traditional apps either return the right answer or throw an error. AI can return a confident, fluent, completely wrong answer.

Cost is per-token, not per-request

Token-level instrumentation is mandatory. A request that costs $0.001 vs $0.10 looks identical without it.

Pipelines are multi-step

RAG, agents, and tool calls mean dozens of components per request. Tracing isn't optional.

Data sensitivity is amplified

Prompts and answers can contain PII. Telemetry itself becomes a data-protection risk.

HACKTIV8

Designing the Telemetry Schema

A canonical event envelope is the contract every component agrees on. Every log, metric, and trace carries the same set of identifying fields — that's how you correlate them later.

Two rules:

- Version your schema (schema_version field).
- Evolve it additively — never remove or rename a field that's already in production.

CANONICAL EVENT ENVELOPE

```
{
  "schema_version": "1.2",
  "trace_id":   "9f2c-...-b81",
  "span_id":    "7a4-...-d12",
  "user_id":    "<hashed>",
  "session_id": "s-2025-...",
  "service":    "search-api",
  "model":      "gpt-4-turbo",
  "model_version": "2024-04",
  "tokens_in":  412,
  "tokens_out": 188,
  "latency_ms": 820,
  "cost_usd":   0.0048,
  "status":     "OK",
  "ts":         "2026-05-10T07:40:11Z"
}
```

HACKTIV8

Trace ID Propagation: The Invisible Thread

Generate one trace_id at the entry point, then carry it through every component. Without it, you can't reconstruct what happened in a single request.

STANDARD: W3C Trace Context

Use the standard traceparent and tracestate HTTP headers. Every modern observability tool understands them, so you don't lock yourself into one vendor.

HACKTIV8

Six Instrumentation Layers

Each layer of an LLM application emits its own signals. Skipping a layer means a blind spot.

01 Edge / API gateway

Request, auth, rate limit, geography.

02 Orchestration

Intent, routing, agent decisions, tool calls.

03 Retrieval

Query, top-k, recall scores, source IDs.

04 LLM call

Model, version, prompt hash, tokens, cost, finish reason.

05 Post-processing

Guardrail verdict, citations, formatting.

06 Client telemetry

Render time, copy events, user feedback.

HACKTIV8

OpenTelemetry: The Vendor-neutral Standard

WHAT IS OTEL?

OpenTelemetry (OTel) is an open-source framework for generating, collecting, and exporting telemetry data.

In plain English:

One SDK to instrument your code, one protocol (OTLP) to ship the data, and you can swap backends anytime without rewriting instrumentation.

Why teams adopt it

Auto-instrumentation

Available for Python, Node.js, Java, Go, .NET — minimal code changes.

Vendor-neutral

Export to Prometheus, Jaeger, Tempo, Datadog, or any OTLP backend.

GenAI conventions

Emerging gen_ai.* attributes standardize LLM telemetry across tools.

Backed by CNCF

Industry standard with broad community and vendor support.

HACKTIV8

Sampling: You Can't Keep Everything

HEAD-BASED SAMPLING

Decide at request entry whether to keep the trace.

- Cheap and simple
- Predictable storage cost
- May miss rare errors

TAIL-BASED SAMPLING

Decide after seeing the full trace.

- Always catches errors
- Catches slow requests
- More infra (collector buffer)

RULE OF THUMB: Always sample 100% of errors and slow requests. Sample 1–5% of normal traffic.

HACKTIV8

Privacy: PII is a Telemetry Problem Too

THE PROBLEM

Prompts contain user secrets. Responses contain user data. If you log them carelessly, your telemetry pipeline becomes a data-leak vector.

Real risk:

A debug log dumps a prompt containing an employee's salary into your shared logging cluster — accessible to 200 engineers.

Five defenses

- Don't log raw prompts/responses by default
- Hash or tokenize user identifiers
- Run a redaction pipeline before storage
- Separate hot path from analytics warehouse
- Set retention aligned with data classification

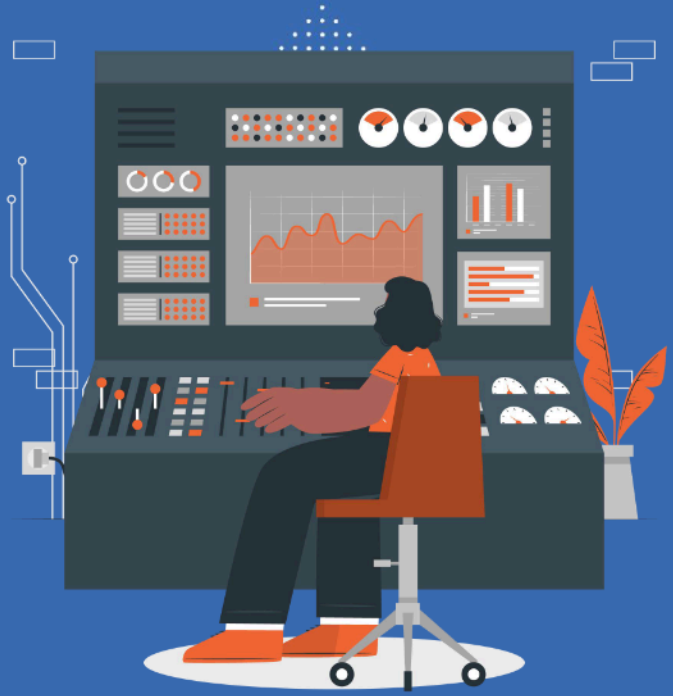
HACKTIV8

Telemetry Design Checklist

Before shipping AI features to production, walk through this list.

- [ ] Canonical event schema documented and versioned
- [ ] All six instrumentation layers covered
- [ ] PII handling reviewed by data governance
- [ ] Dashboards built before launch — not after the first incident
- [ ] trace_id propagated through every component
- [ ] Sampling policy defined and reviewed
- [ ] Telemetry storage cost estimated and budgeted
- [ ] On-call runbook references the right traces and logs

Monitoring Token Usage



HACKTIV8

Why Token Monitoring is Critical

10×	$0.001 vs $0.10	1st
How fast a bad prompt can multiply your bill	Range of per-request cost, same UI	What runaway tokens often signal: an attack

Four reasons to track tokens at every layer

Cost

Tokens directly map to bill — finance needs accuracy.

Latency

More tokens = slower responses; users feel it.

Quota

Vendors throttle on tokens — exhausting them = outage.

Abuse

Sudden spikes often = prompt injection or runaway loops.

HACKTIV8

Token Anatomy Refresher

WHAT GETS COUNTED

Input tokens

System prompt	RAG context	History	User query

Output tokens

Model completion

Reasoning models add hidden "thinking" tokens that you also pay for.

KEY FACTS

- Tokens ≠ words. "Tokenization" is a model-specific process.
- Different models price input vs output tokens differently.
- Cached tokens often cost less — design for cache hits.
- Long context = high token count, even if user query is short.

HACKTIV8

Core Token Metrics to Track

METRIC	TYPE	WHY YOU NEED IT
`tokens_input_total`	Counter	Total prompt tokens — input cost driver.
`tokens_output_total`	Counter	Total completion tokens — usually pricier than input.
`tokens_per_request`	Histogram	Distribution shape — catches bloated prompts.
`cost_usd_total`	Counter	Derived from tokens × price. Single number for finance.
`cache_hit_ratio`	Gauge	High ratio = lower cost, faster responses.
`tokens_per_user_use_case`	Counter	Attribution — who is spending what?

HACKTIV8

Dimensional Tagging: Slice by What Matters

DO TAG BY

- model — different models, different costs
- model_version — track regressions across versions
- environment — dev / staging / production
- use_case — search vs chat vs summarization
- tenant — for multi-tenant cost showback
- region — geographic latency analysis

DON'T TAG BY

- Raw user_id — cardinality explosion
- Full prompt text — privacy and cardinality
- Timestamps as labels — already a built-in axis
- Free-form error messages — use error code instead
- Trace IDs as labels — use exemplars instead

Cardinality is the enemy of Prometheus. Every unique label combination = a new time series.

HACKTIV8

Cost Calculation Pipeline

From raw token counts to a number finance can audit.

1	**Capture token counts at the LLM call**	tokens_in, tokens_out, model, model_version emitted as part of the canonical event.
2	**Pull pricing from a config service**	Never hardcode prices — they change. Cache in memory, refresh hourly.
3	**Compute cost at emit time, not query time**	Doing the multiplication once at write avoids re-deriving at every dashboard load.
4	**Aggregate per minute, hour, day, tenant**	Recording rules in Prometheus, materialized views in your warehouse.
5	**Reconcile with vendor invoice monthly**	If your number is more than 2% off, you have an instrumentation gap.

HACKTIV8

Detecting Token Anomalies

Watch the shape of the change, not just the number — each pattern points to a different root cause.

↑↑

Spike in tokens_per_request

Likely cause: prompt template change, new system prompt, or context bloat from retrieval.

↑→

Input up, output flat

Likely cause: retriever pulling too many or too-long chunks. Check top-k and chunk size.

→↑

Output sustained high

Likely cause: model rambling or missing stop conditions. Tighten max_tokens.

$↑

Cost up, tokens flat

Likely cause: routing change moved traffic to a more expensive model.

HACKTIV8

Budget and Quota Enforcement

Budget burn states

0 – 50 %	Healthy	Normal traffic, no action.
50 – 80 %	Watch	Notify owner. Review forecast.
80 – 100 %	Warn	Soft block low-priority traffic.
>100 %	Halt	Hard block. Fall back to smaller model.

Enforcement mechanisms

Token bucket rate limiter

At the API gateway, per tenant or use case.

Forecast-based alerts

Alert when projected end-of-month spend exceeds budget.

Circuit breakers

Auto-halt non-critical features when budget is at 100%.

Graceful degradation

Route to a cheaper model rather than failing the user.

HACKTIV8

Optimization as an Ops Discipline

Five techniques, measured before and after

Prompt compression	Strip redundancy, deduplicate system instructions.
Context trimming	Rerank retrieved chunks; keep only top-N relevant.
Prompt caching	Cache repeated prefixes (system prompt, few-shots).
Right-size the model	Small model for routing, large for synthesis.
Measure with the same metrics	Same dashboard before and after to prove savings.

HACKTIV8

Token Dashboard: What "Good" Looks Like

12.4 M	$184.20	$0.0023	62 %
Total tokens (24h)	Cost (24h)	Cost / request	Cache hit ratio

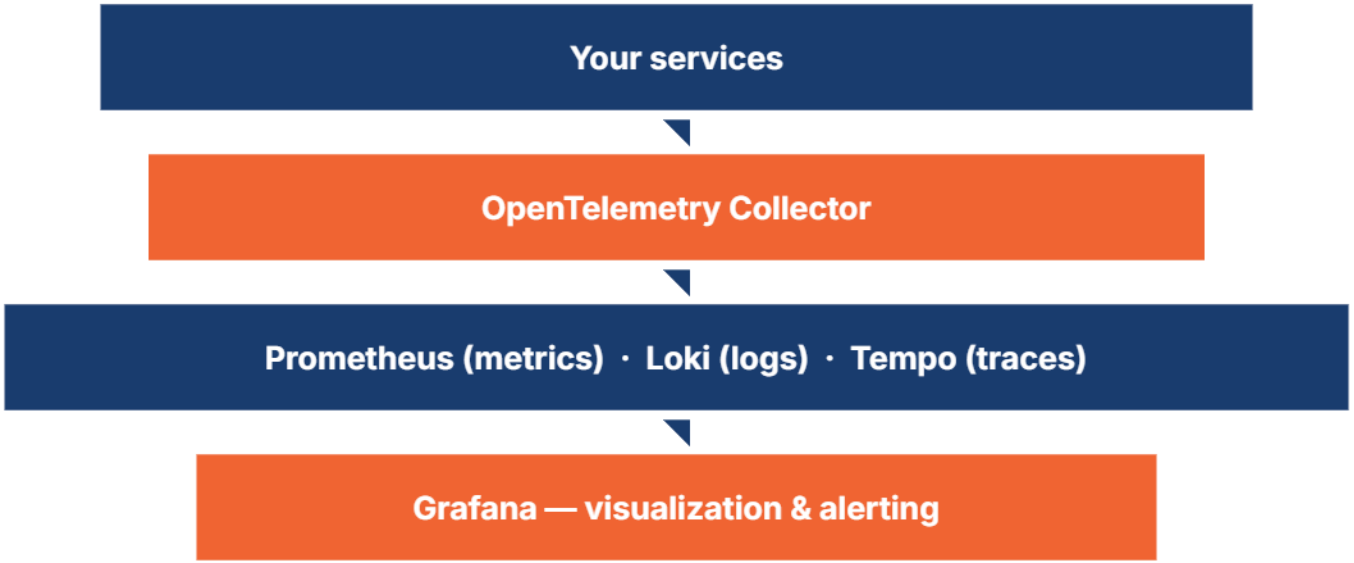
Goal: a single page where finance and engineering agree on the same numbers.

HACKTIV8

Grafana/ Prometheus Style Dashboards

HACKTIV8

The Prometheus + Grafana Stack



Pull-based scraping

Prometheus scrapes /metrics endpoints on a schedule.

PromQL

Powerful query language for time-series data.

Alertmanager

Routes alerts to Slack, PagerDuty, email.

HACKTIV8

Metric Types in Prometheus

Counter	Gauge	Histogram	Summary
Always goes up	*Goes up and down*	*Distribution of values*	*Quantiles, client-side*
`requests_total` `errors_total`	`active_sessions` `queue_length`	`latency_seconds_bucket` `token_count_bucket`	`request_duration_summary`
Use rate() to see throughput.	Read the current value directly.	Use histogram_quantile() for p95.	Use sparingly — can't aggregate.

HACKTIV8

PromQL Essentials: Five Queries to Know

Rough mental model: pick a metric, transform it over time, group by labels.

`rate()`	`rate(requests_total[5m])`	Per-second rate of a counter over a 5-minute window.
`sum by ()`	`sum by (model) (rate(tokens_total[5m]))`	Aggregate across instances, group by a label.
`histogram_quantile ()`	`histogram_quantile(0.95, rate(latency_bucket[5m]))`	Compute the p95 latency from a histogram.
`increase()`	`increase(errors_total[1h])`	Total count over a window — useful for daily volumes.
`alert rule`	`expr: error_rate > 0.05` `for: 5m`	for: clause prevents alerts from flapping on brief spikes.

HACKTIV8

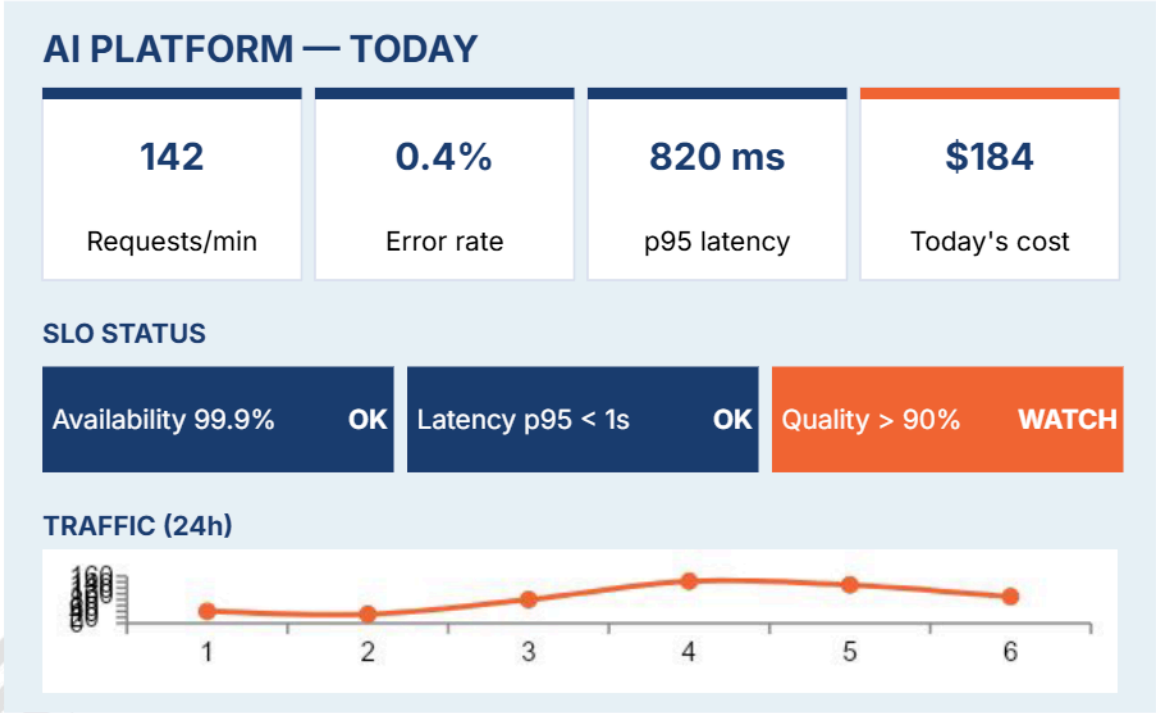
Dashboard Frameworks: USE, RED, Quality

Standard mental models for what to put on a dashboard.

USE	RED	+Q
for infrastructure	*for services*	*for AI (added)*
• Utilization — % of resource used • Saturation — work waiting in queue • Errors — count of failures	• Rate — requests per second • Errors — failed requests • Duration — latency distribution	• Quality — groundedness, refusal rate • Drift — distribution shifts • User feedback — thumbs / CSAT

HACKTIV8

Dashboard 1: Executive Overview



Design rules

- Single screen — no scrolling.
- Color-coded SLO status (visible at a glance).
- 4–6 headline KPIs — no more.
- Audience: leadership, on-call lead.
- Updated every 30s; reviewed daily.

HACKTIV8

Dashboard 2: Service-level (per Microservice)

One per service. Owned by the team that runs that service.

RED metrics

Rate, Errors, Duration — the heartbeat of the service.

Dependency health

Downstream LLM provider, vector DB, cache, queues.

Resource saturation

CPU, memory, GPU (if applicable), connection pools.

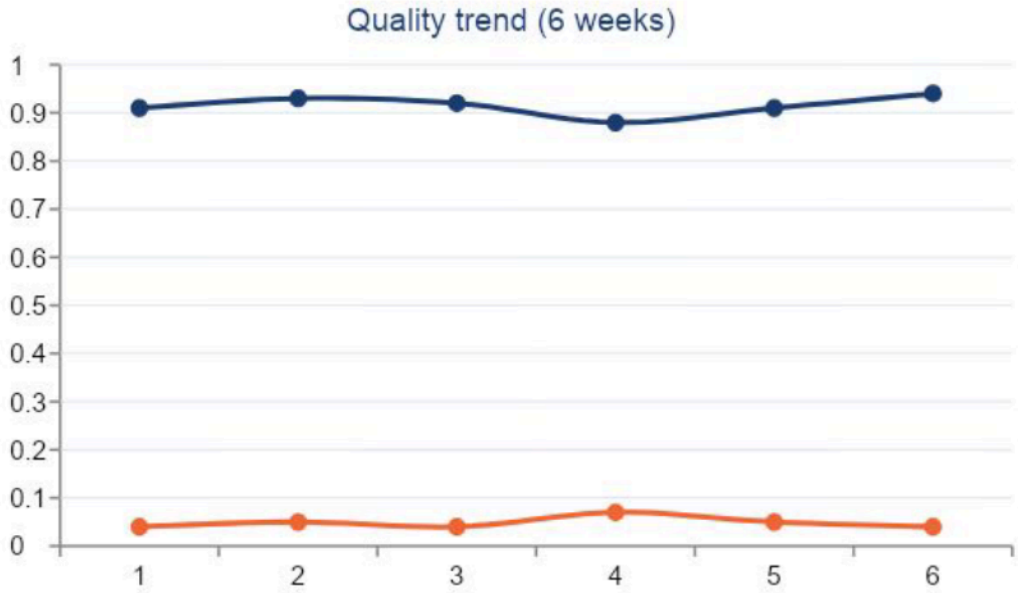
Deploy markers

Annotate every release on the timeline. Easy diff after incidents.

Owner: the team running this service. Reviewed weekly during ops sync.

HACKTIV8

Dashboard 3: LLM Quality

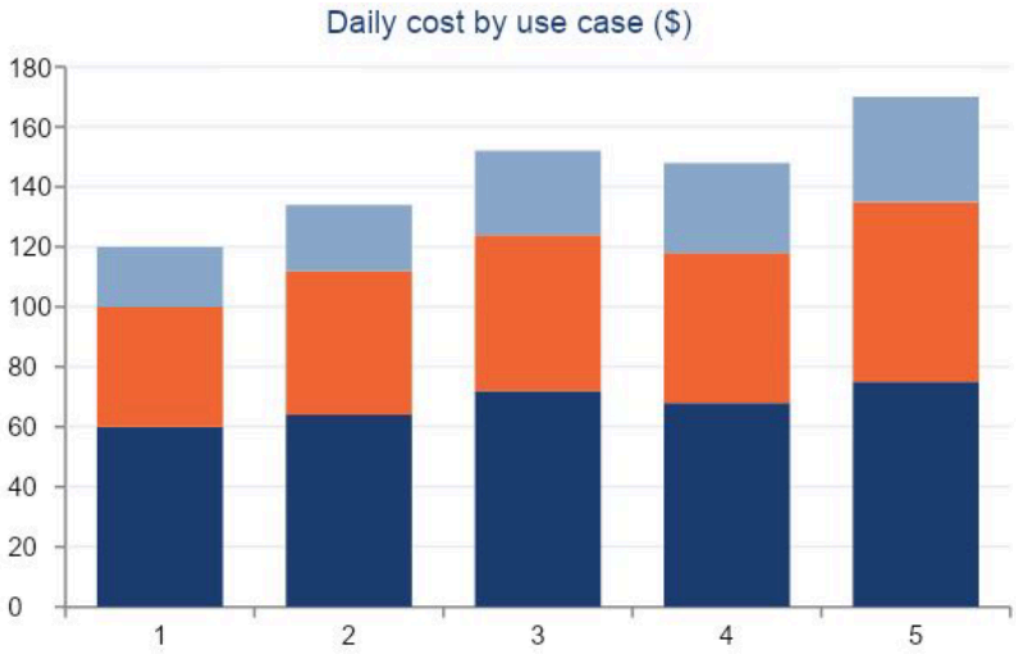


Panels to include

- Groundedness score over time
- Refusal / guardrail-trigger rate
- Hallucination flags from offline evals
- User feedback distribution (thumbs)
- Drift indicators (query length, embeddings)

Quality is the metric that tells you when the model is wrong before users do. Without it, you only learn from complaints.

Dashboard 4: Token & Cost



Monthly budget burn

$5,200 of $8,000 · 65%

Panels to include

- Tokens in/out per minute
- Cost burn-down vs. monthly budget
- Top use cases / tenants by cost
- Cache hit ratio trend
- End-of-month forecast line

HACKTIV8

Alerting Best Practices

THE GOLDEN RULE

Alert on symptoms — what users feel — not on causes.

Why?

Cause-based alerts ("CPU high") fire constantly even when users are happy. Symptom alerts ("users seeing errors") wake you up only when it matters.

Five practices to follow

Multi-window burn rates

Combine fast-burn and slow-burn (Google SRE pattern).

Runbook URL on every alert

First line of the alert: how to fix it.

Clear on-call rotation

One owner, paged primary + backup.

Suppression for maintenance

Silence known windows — avoid alert fatigue.

Quarterly review

Kill noisy alerts. Add ones for missed incidents.

HACKTIV8

Operational Logging Best Practice

HACKTIV8

Logs vs. Metrics vs. Traces: When to Use Which

Metrics	Logs	Traces
What is happening?	***What exactly happened?***	***How did it flow?***
At scale, cheap to query.	Detailed, per-event.	Across multiple services.
Aggregated numbers over time.	*One event = one record.*	*End-to-end request path.*

Use all three. Logs alone don't scale. Metrics alone lack detail. Traces alone don't tell you what happened in the code.

HACKTIV8

Structured Logging is Non-negotiable

UNSTRUCTURED · AVOID

```
[2026-05-10 07:40:11] User abc123 made a
request to /search and got 200 OK in 820ms
with 412 input tokens
```

Why this is bad:

- Unparseable at scale
- Can't filter by field
- Format drift across services

STRUCTURED · PREFERRED

```
{
  "ts": "2026-05-10T07:40:11Z",
  "level": "INFO",
  "service": "search-api",
  "trace_id": "9f2c-...-b81",
  "user": "<hashed>",
  "endpoint": "/search",
  "status": 200,
  "latency_ms": 820,
  "tokens_in": 412
}
```

One event per line. Required keys: ts, level, service, trace_id.

HACKTIV8

Log Levels: When to Use Each

TRACE	Very fine-grained. Dev only. Off in prod.
DEBUG	Diagnostic. Off in prod by default; toggle for incident.
INFO	Normal lifecycle events. Request received, completed.
WARN	Recoverable anomaly. Retry succeeded, fallback used.
ERROR	Failure with user impact. Always logged.
FATAL	Process can't continue. Exit and let supervisor restart.

Avoid "INFO everything". It costs money to store and hides real signals.

HACKTIV8

Log This: Never Log That

DO LOG

- Request entry: trace_id, endpoint, params summary
- Retrieval: query hash, top-k, scores, source IDs
- LLM call: model, version, token counts, latency
- Guardrail: verdict, rule triggered (not full prompt)
- Errors: full stack + correlation IDs
- Audit: who, what, when, on which record

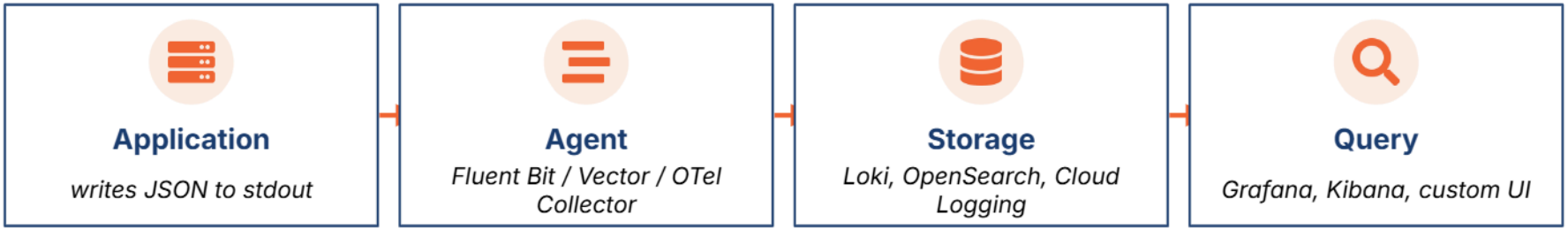
NEVER LOG

- Raw prompts and completions (PII risk)
- API keys, JWTs, passwords, credentials
- Full retrieved documents (log IDs and hashes)
- Customer-identifying data outside controlled stores
- Sensitive personal data (medical, financial, legal)
- Anything you'd be uncomfortable seeing in a leak

HACKTIV8

Aggregation and Storage

Log pipeline



HOT TIER

7 to 14 days. Fast search. Used during incidents.

Examples: Loki, Elasticsearch with hot indices.

COLD / ARCHIVE TIER

90+ days, sometimes years. Slower. Audit and compliance.

Examples: S3 / GCS object storage with lifecycle rules.

HACKTIV8

Correlating Logs, Metrics, and Traces

trace_id is the universal join key. Every modern observability tool understands it.

OPERATIONAL STORY

Five clicks from "alert fired" to "I know why". That's the bar. If your stack takes more than five clicks, your tooling needs work — not your team.

HACKTIV8

Key Takeaways

Telemetry is a first-class system

Design it before you ship features. Schemas, trace IDs, and PII handling are non-negotiable foundations.

Tokens are money — govern them

Track tokens at every layer. Tag with the right dimensions. Enforce budgets with circuit breakers.

Dashboards exist for an audience

Executive, service-level, quality, cost — each has different consumers and different decisions.

Logs are evidence

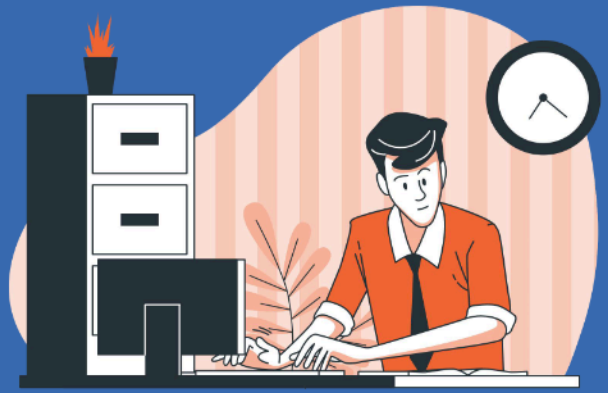
Structure them. Redact PII. Retain for audit. Correlate to traces with a shared key.

Observability is the foundation

You can only operate, optimize, and trust an AI system that you can see end-to-end.

HACKTIV8

Guided Hands-on



HACKTIV8

Four Labs: Building on Each Other

01	02	03	04
			
Tracing	**Token & cost metrics**	**Dashboard**	**Logs + correlation**
Wrap the 3-stage pipeline with OpenTelemetry. One parent + 3 child spans, one shared trace_id.	Add Prometheus counters and histograms. Tag with model and use_case for attribution.	Pull metrics into pandas, render a 2×2 dashboard with Plotly. Headline KPIs + breakdowns.	Structured JSON logs with auto-injected trace_id. Walk an injected incident in 5 clicks.

HACKTIV8

Lab 01: OpenTelemetry Tracing

WHAT YOU BUILD

- TracerProvider with service-name resource
- ConsoleSpanExporter (so spans print in Colab)
- handle_query() wraps retrieve / llm_call / guardrail
- Each span carries useful attributes (token counts, source IDs, verdict)

CHECKPOINT

4 spans printed. All share the same trace_id. Children point at the parent via parent_id.

Span tree shape

All four spans share the same trace_id.

HACKTIV8

Labs 02 & 03: Metrics and The Dashboard

LAB 02 · METRICS

- Counters: tokens_in/out, cost, requests
- Histograms: latency, tokens-per-request
- Labels: model, use_case (low cardinality)
- Cost computed at emit time, not query time
- Generate 100 mixed-traffic requests

LAB 03 · DASHBOARD

- Pull metrics into a pandas DataFrame
- Compute KPIs: requests, tokens, cost, error rate
- Plotly 2×2 grid mimics Grafana layout
- Panels: cost by model, by use case, latency
- Read three answers in under 30 seconds

In production: swap the in-memory registry for a /metrics scrape endpoint and you have Grafana over real Prometheus.

HACKTIV8

Lab 04: Logs, Correlation, and a 5-click Incident

Inject a slow vector-DB. Walk from the alert to the root cause using your own stack.

WHY THIS LAB MATTERS

In production at 2 AM, this is the only thing that matters. If the path takes more than 5 clicks, your tooling needs work — not your team.

HACKTIV8

Logging, Monitoring, and Analytics

🎯 Learning Goals

By the end of this session, participants will be able to:

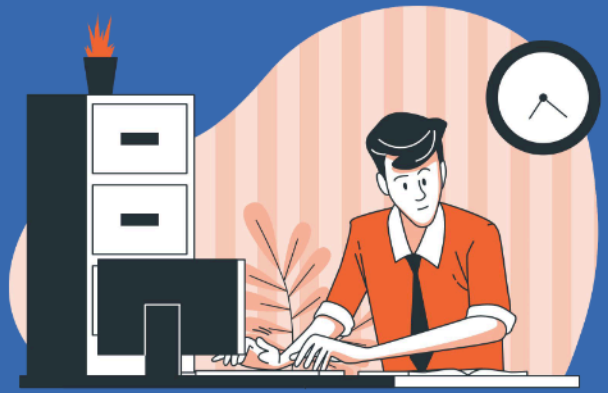
- Instrument an AI pipeline with OpenTelemetry tracing
- Define Prometheus metrics with proper labels and cost attribution
- Build a dashboard from raw metrics
- Emit structured logs with auto trace_id and PII protection
- Walk an incident from alert to root cause

🛠️ Setup

Participants can use:

- Google Colaboratory: https://drive.google.com/file/d/1B986c51bX4opriXnsy7esvG8Vosavr_J/view?usp=sharing

Assignment



HACKTIV8

Six Deliverables in One Notebook

01 Telemetry & tracing

1 parent + 3 child spans, useful attributes.

02 Token & cost metrics

Counters + histograms with ≥3 thoughtful labels.

03 One audience dashboard

Pick exec / on-call / finance — design 4–6 panels.

04 Structured logs + PII

JSON, auto trace_id injection, no raw queries.

05 Alert + runbook

Symptom-based, with 5-step on-call instructions.

06 Incident report

Trainer injects fault. You document the path.

HACKTIV8

Common Pitfalls and How to Avoid Them

Hardcoded API key

Even commented-out keys count. Always Colab Secrets.

user_id as a metric label

Cardinality explosion. Use it as an attribute on spans, not a metric label.

Logging the full prompt

PII leak. Log a fingerprint hash + length.

Cause-based alerts

"CPU high" wakes you up when users are happy. Alert on what users feel.

Generic dashboard

If it answers everyone's question, it answers nobody's. Pick one audience.

Computing cost at query time

Prices change. Compute at emit time, store the dollar figure.

HACKTIV8

Submission Checklist: Before You Click Submit

Run through this list. If any box is unchecked, fix before submitting.

- [ ] No hardcoded secrets anywhere — search the file for any literal key
- [ ] All TODO comments resolved
- [ ] Notebook runs top to bottom without manual edits
- [ ] Part 1 prints 4 spans with one shared trace_id
- [ ] Part 2 metrics use only the labels you justified
- [ ] Part 3 dashboard names its target audience clearly
- [ ] Part 4 logs contain query fingerprints, never raw text
- [ ] Part 5 alert is symptom-based and has a 5-step runbook

HACKTIV8

Deliverables: Submit All Three

The notebook (.ipynb)

Every cell executed, all output visible, no leftover # TODO comments. You can follow example here: Session 39 Assignment_Logging, Monitoring, and Analytics.ipynb

The report (3–5 pages, PDF)

Written for a non-technical reader. Cover what you did, what you found, what you recommend.

The presentation (5 slides)

Clean summary suitable for the next session. You will present briefly to peers.

HACKTIV8

Thank You

Hacktiv8id

Hacktiv8id

Hacktiv8

Hacktiv8 Indonesia